End-of-Studies Project Report

Design and Implementation of an End-to-End Satellite Imagery Acquisition and Processing Pipeline

NimbusChain Pipeline

Submitted in partial fulfillment of the requirements for the Software Engineering Degree

Major: Computer Engineering

Promotion 2025 / 2026

Internship carried out within Oracle Morocco R&D Center - Digital Government

Written by	Mr. Mehdi Dinari
Company mentor	Mr. Hamza El Makrini
School tutor	Mr. Abdelhak Boulaalam
Defense date	.....

Jury Members: Mr. Hamza El Makrini, Mr. Abdelhak Boulaalam, and invited ENSAF professors

Dedication

This work is dedicated to my family, my supervisors and my friends, whose support, patience and encouragement accompanied me throughout this internship and engineering journey.

Their trust gave me the motivation to overcome technical difficulties, learn from each iteration and complete this project with discipline and curiosity.

Acknowledgments

I would like to express my sincere gratitude to my company mentor, Mr. Hamza El Makrini, for his guidance, availability and technical feedback throughout this internship. His advice helped me understand the project environment, improve the quality of my work and approach engineering decisions with more rigor.

I also thank my school tutor, Mr. Abdelhak Boulaalam, for his academic guidance and constructive feedback. His support contributed to the organization of this report and to the alignment of the project with the expectations of an end-of-studies engineering work.

My sincere thanks go to my managers, Mr. Faical Tabouti and Mr. Sam Batterman, for the supportive environment they provided and for their availability whenever I needed assistance or clarification.

I extend my appreciation to the Oracle Digital Government team and to the administration, professors and staff of ENSA Fès for the knowledge, professional values and learning conditions they provided during my training.

Finally, I thank the members of the jury for accepting to evaluate this work, as well as my family and friends for their constant encouragement.

Abstract

This report presents the design and implementation of an end-to-end satellite imagery acquisition and processing pipeline developed during an internship within Oracle's Digital Government context. The project addresses a practical operational need: analysts need to define an area of interest, preview available scenes, launch provider downloads, normalize the outputs into analysis-ready structures and monitor the full execution lifecycle from a single interface.

The implemented platform is service-oriented. It combines a FastAPI backend, background workers, a Streamlit user interface, a Zarr conversion service and a mask service for cloud and water processing. The pipeline supports provider preview, asynchronous job orchestration, provider-aware download coordination, conversion of raw scenes to normalized scene-level Zarr stores, optional cube generation and execution in local or cloud environments.

The resulting system improves traceability, reduces manual intervention and provides a reusable basis for operational Earth-observation workflows. The validation phase also highlights a key engineering constraint: precise water masking with OmniWater requires an accelerator-backed runtime for full scenes, while the CPU fallback remains useful only for smaller workloads.

Keywords: Remote Sensing, Satellite Imagery, Zarr, Microservices, Cloud Mask, Water Mask, Oracle Cloud Infrastructure.

Résumé

Ce rapport présente la conception et la mise en oeuvre d'une plateforme de téléchargement et de traitement d'images satellitaires développée dans le cadre d'un stage au sein du contexte Digital Government d'Oracle. Le projet répond à un besoin opérationnel concret: permettre à un utilisateur de définir une zone d'intérêt, de prévisualiser les scènes disponibles, de lancer les téléchargements depuis plusieurs fournisseurs, de normaliser les sorties dans un format exploitable et de suivre tout le cycle d'exécution depuis une interface unique.

La solution proposée repose sur une architecture orientée services composée d'un backend FastAPI, de workers d'arrière-plan, d'une interface Streamlit, d'un service de conversion vers Zarr et d'un service de masquage nuage/eau. La plateforme prend en charge la prévisualisation des produits, l'orchestration asynchrone des jobs, la coordination des téléchargements, la conversion des scènes brutes vers des Zarr normalisés, la construction optionnelle de cubes temporels et l'exécution en local ou sur Oracle Cloud Infrastructure.

Les résultats montrent que l'architecture améliore la traçabilité, limite les manipulations manuelles et facilite la réutilisation des sorties. Le rapport met également en évidence une limite importante: pour obtenir un masque d'eau précis et rapide sur des scènes complètes, le modèle OmniWater doit être exécuté sur un runtime accéléré, tandis que le mode CPU doit rester un mécanisme de secours.

Mots-clés: télédétection, imagerie satellitaire, Zarr, microservices, masque nuage, masque eau, Oracle Cloud Infrastructure.

Table of Contents

Confidential - Oracle Restricted \Employees Only

List of Figures

List of Tables

Chapter 1 - Presentation of Oracle and the Project Environment

Introduction

This chapter presents the professional environment in which the internship was carried out. It introduces Oracle Corporation, the Oracle Morocco Research and Development Center, the Digital Government group and the organizational context that framed the project.

Oracle Corporation

Oracle Corporation is a multinational technology company founded in 1977 by Larry Ellison, Bob Miner and Ed Oates. The company first became known through Oracle Database, a relational database management system that became one of the foundations of modern enterprise information systems.

Over time, Oracle expanded its portfolio to include enterprise applications, cloud infrastructure, middleware, hardware systems, developer tools, consulting services and advanced data platforms. The acquisition of Sun Microsystems in 2010 also strengthened Oracle's position in major technologies such as Java and Solaris.

Oracle Morocco R&D Center

The Oracle Morocco Research and Development Center is one of Oracle's strategic engineering centers in the EMEA region. Located in Casablanca, it brings together software engineers, data specialists, project managers and research-oriented teams working on a broad range of Oracle products and initiatives.

The center contributes to domains such as cloud computing, data platforms, artificial intelligence, cybersecurity, analytics and digital public services. It also supports internships, graduate recruitment initiatives and collaboration with Moroccan engineering schools.

Digital Government Context

The project was carried out within the Digital Government context. This environment focuses on digital platforms that help public-sector actors use data more effectively, improve service delivery and support operational decision-making.

Geospatial information is particularly relevant in this context because it can support agriculture, land-use observation, water monitoring, infrastructure planning and environmental reporting. However, transforming satellite products into reliable digital services requires more than downloading images. It requires reproducible pipelines, runtime visibility, normalized outputs and deployment strategies adapted to heavy geospatial workloads.

Project Motivation

The internship project addresses the need for a reusable satellite-processing platform. Existing workflows are often fragmented across provider portals, local scripts and manual processing steps. This fragmentation makes it difficult to follow progress, recover from failures, preserve output lineage and expose the workflow to non-expert users.

The goal of the platform is therefore to transform a scattered workflow into an operational chain that supports product discovery, asynchronous execution, normalized Zarr outputs, optional cube building and cloud or water masking.

Conclusion

This chapter introduced Oracle and the professional context of the internship. It also clarified why satellite imagery processing is relevant for Digital Government use cases and why an operational pipeline is needed before downstream geospatial analysis can be performed reliably.

Confidential - Oracle Restricted \Employees Only

Chapter 2 - Domain Background and Key Concepts: Remote Sensing

Introduction

This chapter introduces the remote sensing concepts required to understand the NimbusChain pipeline. The objective is not to provide an exhaustive scientific presentation of Earth observation, but to explain the data structures, satellite missions and preprocessing needs that directly influenced the engineering design.

Earth Observation Context

Earth observation consists of acquiring information about the Earth's surface using remote sensing technologies, especially satellites. For agricultural and environmental applications, satellite imagery makes it possible to observe large areas repeatedly and objectively.

The platform focuses on workflows where users select an area of interest, choose dates and providers, inspect candidate products and transform raw scenes into outputs that are easier to analyze. This requires both geospatial understanding and robust software orchestration.

Satellite Product Families

The project mainly considers Sentinel-2 and Landsat 8/9 products. These missions are widely used in optical Earth observation and are relevant for vegetation monitoring, land-cover analysis, water observation and environmental reporting.

Mission	Main characteristics	Role in the project
Sentinel-2	Multispectral optical mission with bands at 10 m, 20 m and 60 m resolution.	Used as a reference target structure for high-resolution optical analysis and Zarr normalization.
Landsat 8/9	Long-running Earth observation program with optical, panchromatic and thermal information.	Used as an institutional provider source, especially through USGS access and Sen2Like harmonization.
Sen2Like output	Processing output that makes Landsat products closer to Sentinel-like structures.	Allows downstream stages to rely on a more homogeneous representation across providers.

Table 2.1: Main satellite product families handled by the pipeline.

Spectral Bands and Interpretation

Satellite sensors capture information in several spectral bands. Unlike ordinary RGB images, multispectral products include visible, near-infrared, shortwave infrared and sometimes thermal bands.

These bands carry complementary information about vegetation, water, soil, clouds and surface conditions.

Band category	Interpretation	Typical downstream use
Visible bands	Blue, green and red bands approximate human visual perception.	RGB preview, visual inspection, land-cover distinction.
Near-infrared	Healthy vegetation strongly reflects NIR energy.	Vegetation condition, biomass and crop vigor analysis.
Red-edge	Sensitive to subtle vegetation and chlorophyll variations.	Agriculture-oriented monitoring and stress detection.
SWIR	Useful for moisture, soil and vegetation condition analysis.	Water-related indices, surface moisture and stress analysis.
Quality layers	Provider-generated or model-generated scene quality information.	Cloud, shadow, snow, water or invalid-pixel filtering.

Table 2.2: Spectral band categories used in remote sensing interpretation.

Tile Systems and Spatial Organization

Satellite products are not distributed as one continuous global image. Sentinel-2 uses the MGRS tiling system, while Landsat products are organized through the WRS-2 path/row logic. A single user area may therefore intersect one or several tiles, which affects search, download, conversion and cube generation.

Why Preprocessing Is Needed

Raw products are heterogeneous. They differ in file naming, metadata, spatial resolution, band organization and provider-specific conventions. They may also include clouds, shadows or other artifacts that reduce direct analytical value.

Preprocessing in NimbusChain therefore includes provider-aware download coordination, Sen2Like normalization when needed, conversion to Zarr, optional cloud and water masking and optional temporal cube building. These stages transform raw provider outputs into structured artifacts that can be reused by later analytical services.

Confidential - Oracle Restricted \Employees Only

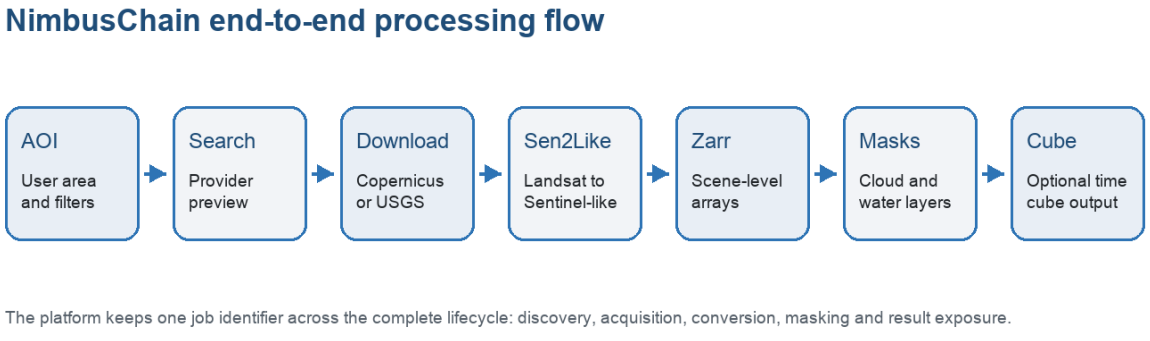

Figure 2.1: NimbusChain end-to-end processing flow.

Conclusion

Remote sensing data is rich, but operationally complex. Understanding missions, bands, tile systems and preprocessing requirements explains why the project needs a modular pipeline rather than a simple downloader.

Confidential - Oracle Restricted \Employees Only

Chapter 3 - General Project Context

Project Description

NimbusChain is an end-to-end satellite imagery acquisition and preparation platform. Its role is to move from user intent to analysis-ready artifacts through a traceable workflow: area selection, provider preview, job submission, download, normalization, masking and result exposure.

The project is not limited to downloading data. Each stage produces structured outputs that can be inspected, persisted and reused. This is important because operational Earth-observation workflows often require several scenes, several providers and several downstream processing steps.

Problem Statement

The initial problem was the absence of a unified operational pipeline able to manage satellite imagery from preview to analysis-ready outputs. Existing approaches depended on fragmented tools, manual provider interactions and ad hoc scripts. This made it difficult to track progress, recover from interruptions, control throughput and preserve lineage.

Project Objectives

- Provide a user interface for AOI definition, product preview and job monitoring.
- Automate the transition from product selection to processed outputs.
- Execute long-running operations asynchronously through background workers.
- Normalize downloaded products into structured Zarr stores.
- Preserve satellite bands, metadata and auxiliary layers required by masks and analysis.
- Support cloud and water mask generation as explicit pipeline stages.
- Prepare outputs for optional temporal cube construction.
- Keep the codebase modular and extensible for future providers and services.

Architecture Overview

The architecture follows a service-oriented design. The user interface handles interaction; the API handles validation, state and public endpoints; the worker executes long-running tasks; the Zarr service performs conversion and cube operations; and the mask service enriches generated Zarr stores with quality layers.

Component	Technology	Main responsibility
User interface	Streamlit	Collect inputs, preview products, submit jobs and display execution status.
Public API	FastAPI	Expose job, artifact, health, provider, conversion and masking endpoints.

Worker runtime	Python asynchronous worker	Claim jobs, coordinate downloads and drive the pipeline lifecycle.
Zarr service	FastAPI service	Convert raw scenes to Zarr and build optional cubes.
Mask service	FastAPI service	Apply cloud and water masks on existing Zarr outputs.
Persistence	SQLite or MongoDB	Store jobs, events, artifacts, state transitions and worker heartbeats.

Table 3.1: High-level architecture of the platform.

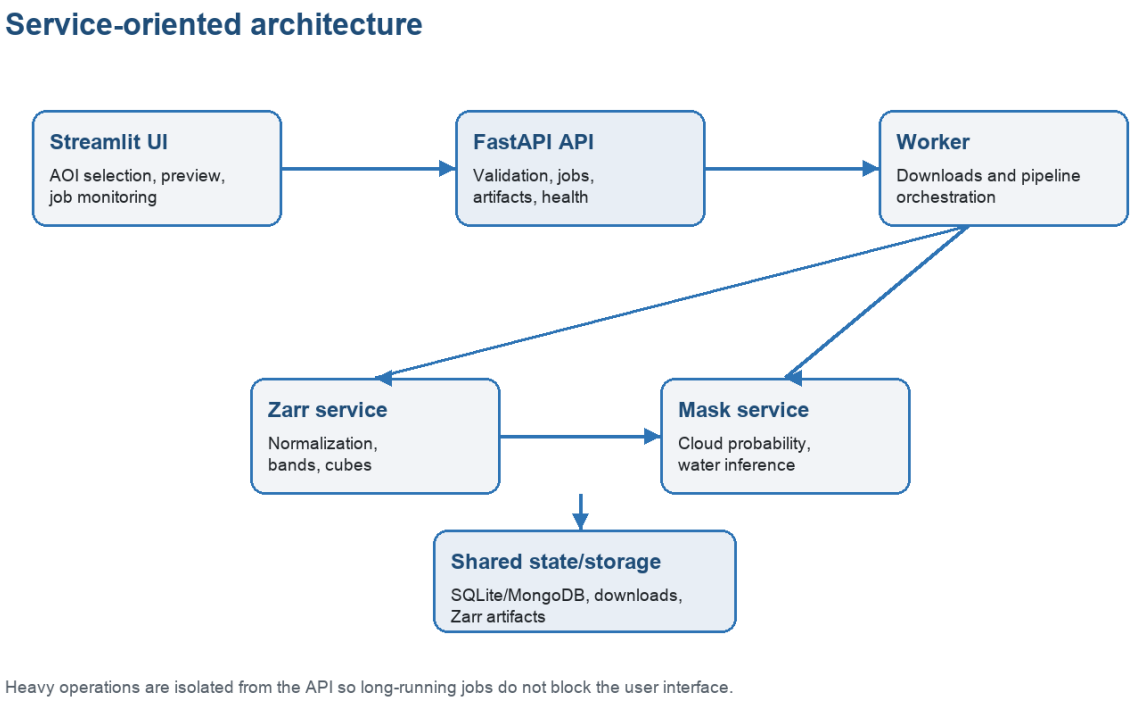

Figure 3.1: Service-oriented architecture.

Why Modularity Is Essential

Modularity is a practical requirement for this project. The platform handles user interaction, provider communication, job orchestration, geospatial conversion, inference-based masking and artifact management. Mixing these responsibilities into one monolithic script would make the system fragile and difficult to evolve.

By separating services, the platform improves maintainability, fault isolation, testability and deployment flexibility. It also makes it possible to move selected heavy stages, such as water masking, to a specialized runtime without redesigning the entire system.

Conclusion

This chapter described the project as a modular satellite data pipeline platform. The next chapter focuses on the conception choices that organize these requirements into a coherent technical design.

Chapter 4 - Conception and Architecture

Needs Analysis

The main user need is to launch and monitor a complete satellite-processing workflow without manually coordinating provider portals, local scripts and post-processing tools. The system must expose a clear operational surface while preserving enough technical detail for debugging and validation.

Functional Requirements

- Search and preview satellite products from supported providers.
- Accept an area of interest, date range, provider and product configuration.
- Submit asynchronous jobs and expose their state, events and artifacts.
- Download selected products while respecting provider constraints.
- Run Sen2Like when Landsat harmonization is required.
- Convert raw or normalized scenes into Zarr stores with preserved bands and metadata.
- Apply cloud and water masks when requested.
- Build scene-level or grouped time cubes when the selected workflow requires it.

Non-Functional Requirements

- Modularity: each runtime component must have a clear responsibility.
- Scalability: heavy execution should be isolated from the UI and API.
- Observability: jobs must expose progress, stage timings, errors and artifacts.
- Reliability: partial failures must be visible and recoverable.
- Portability: the stack must support local development and cloud deployment.
- Extensibility: new providers, masks and output formats should be addable without rewriting the core.

Control Plane and Execution Plane

The control plane is represented by the FastAPI service. It receives user intent, validates requests, creates jobs, exposes state and keeps the UI responsive. The execution plane is represented by workers and processing services that perform downloads, conversions and inference-heavy tasks.

This separation is important because satellite workflows are long-running by nature. A download or mask generation stage can take several minutes, and the API must remain available during that time.

Data Model and Artifact Lineage

The platform relies on persistent job records and artifacts. Instead of returning large raster outputs directly through HTTP payloads, services exchange references such as job identifiers, source URIs, output Zarr URIs, scene identifiers and acquisition timestamps.

This design reduces coupling between services and allows the UI to display a coherent timeline of the pipeline, including generated outputs and stage-level errors.

Confidential - Oracle Restricted \Employees Only

Placement of Masking in the Workflow

Masking is designed as a post-conversion stage that enriches an existing Zarr store. This choice keeps raw acquisition, data normalization and mask inference separated. It also allows cloud and water masks to be recomputed or skipped without repeating the full download stage.

The water mask stage has a specific runtime constraint: the accurate OmniWater model is significantly heavier than simple threshold-based approaches. For full-scene execution, the design must therefore support an accelerator-backed placement while keeping a CPU fallback for small scenes and tests.

Conclusion

The conception phase defines the project as a traceable multi-service workflow. This design makes the pipeline easier to operate and prepares it for later optimization, especially for compute-heavy stages such as water masking.

Confidential - Oracle Restricted \Employees Only

Chapter 5 - Implementation of the Proposed Solution

Implementation of the User Interface

The user interface is implemented with Streamlit. It provides the operational entry point for preparing requests, selecting provider parameters, previewing candidate products, submitting jobs and inspecting results. The UI acts as a client of the backend API and does not perform heavy processing directly.

Implementation of the API Layer

The API layer is implemented with FastAPI. It exposes endpoints for job creation, job inspection, provider health, worker status, artifact discovery, conversion requests and mask operations. It centralizes validation and keeps the contract between UI and execution services stable.

Worker and Job Execution Logic

The worker is responsible for claiming queued jobs and executing the pipeline. It records events, updates state transitions, coordinates downloads and calls downstream services. This makes the processing lifecycle persistent and observable even when individual stages are long-running.

Zarr Conversion Workflow

The Zarr service converts raw or normalized products into structured arrays. The conversion preserves imagery bands, auxiliary layers, spatial context and provenance metadata so that downstream services can access the data consistently. Zarr was selected because its chunked multidimensional structure is suitable for large raster data and cloud-native access patterns.

Masking Workflow

The mask service applies cloud and water masks on existing Zarr stores. Cloud masking produces both binary cloud layers and probability information when the selected backend supports it. Water masking uses OmniWater for model-based inference when the dependency and runtime are available.

A key implementation detail is the separation between output resolution and model inference export size. The platform can keep 512 x 512 output chunks while exporting larger model tiles for OmniWater. This reduces excessive per-tile overhead without changing the final Zarr chunking strategy.

Technology Stack

Category	Technology	Role
Backend	FastAPI, Uvicorn	API endpoints, service contracts and health checks.
Frontend	Streamlit	Interactive workflow preparation and monitoring.
Execution	Python async workers	Background job processing and orchestration.

Geospatial	Rasterio, Xarray, Zarr, PyProj	Raster reading, array representation and spatial metadata handling.
Data validation	Pydantic	Typed request, response and configuration models.
Persistence	SQLite or MongoDB	Job records, events, artifacts and worker state.
Masking	OmniCloudMask, OmniWaterMask	Model-based cloud and water mask generation.
Deployment	Docker, Podman, OCI	Local container stack and cloud-oriented execution.

Table 5.1: Main technology stack used by the project.

Artifact Management

Each major stage registers artifacts so that results can be inspected after execution. Raw downloads, Sen2Like outputs, scene-level Zarr stores, mask layers and cubes are linked to the same job lifecycle. This improves traceability and makes debugging more efficient.

Conclusion

The implementation translates the conceptual architecture into concrete services and workflows. The result is a modular platform where acquisition, conversion, masking and monitoring can evolve independently while remaining connected by shared contracts.

Chapter 6 - Validation, Results and Discussion

Validation Strategy

The platform was validated through iterative end-to-end tests. Each test follows the same operational chain: provider search, download, Sen2Like normalization when required, Zarr conversion, optional cloud mask, optional water mask and final artifact inspection.

Observed Local Results

Local validation showed that search, download, Sen2Like, Zarr conversion and cloud masking can execute through the stack. The water mask stage exposed the most important runtime limitation: in the current local Podman environment, the OmniWater model runs on CPU only, which makes full-scene inference too slow for comfortable interactive testing.

Stage	Observed behavior	Interpretation
Search	Completed quickly during local tests.	Provider metadata discovery is not the bottleneck.
Download	Depends mainly on provider speed and scene size.	Large Landsat products naturally dominate network and disk time.
Sen2Like	Works but remains significant on full scenes.	The stage should not be duplicated and should feed downstream Zarr directly.
Zarr conversion	Completed in a reasonable time after normalization.	Preserving bands and metadata is more important than aggressive simplification.
Cloud mask	Acceptable runtime and useful output layers.	The cloud path can remain local for many test scenarios.
Water mask	Accurate model is slow on CPU-only Podman.	Needs accelerator placement for precise and fast full-scene execution.

Table 6.1: Local validation observations.

Water Mask Issue and Proposed Solution

The water mask problem is not caused by Sen2Like and should not be solved by replacing the model with a heuristic. The accurate OmniWater path is model-based, but the current local container runtime does not expose a usable accelerator. As a result, the model falls back to CPU execution, which is precise but too slow on complete scenes.

The recommended solution is to keep the OmniWater model logic unchanged and move its execution to an accelerator-backed runtime. Two practical placements are possible: a remote OCI GPU mask service

connected to the same pipeline contracts, or a host-native macOS Apple MPS runner if the dependencies are installed outside Podman and shared Zarr paths are available.

The CPU mode should remain a fallback. It can use large model export tiles, for example 4096 pixels, while keeping final 512 x 512 output chunks. This avoids generating hundreds of tiny model inputs, but it cannot replace GPU or MPS acceleration for full operational scenes.

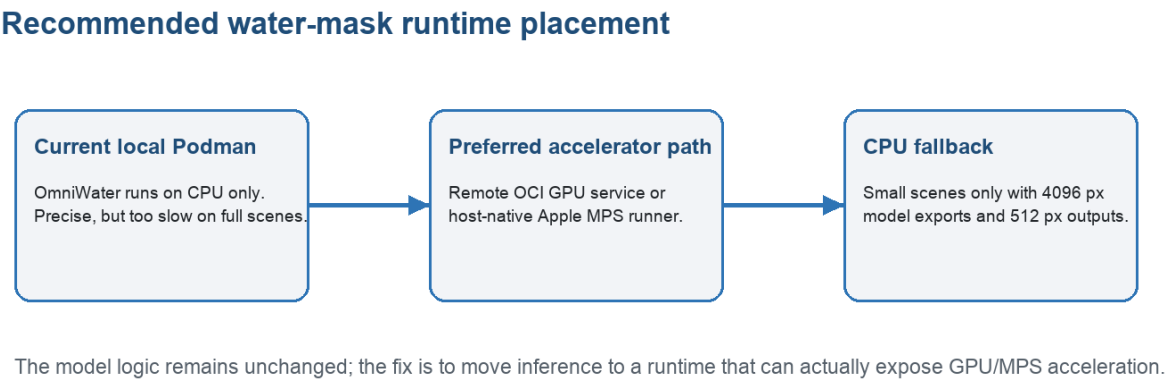

Figure 6.1: Recommended water-mask runtime placement.

Strengths of the Proposed Solution

- The pipeline exposes a coherent end-to-end workflow rather than isolated scripts.
- The service-oriented architecture improves maintainability and deployment flexibility.
- Zarr outputs make downstream access more structured and reusable.
- Cloud and water masks are explicit artifacts that enrich existing datasets.
- Stage-level timing and error reporting make operational debugging easier.

Current Limitations

- Full-scene processing remains heavy because satellite products are large by nature.
- Precise water masking requires accelerator-backed inference for production-like scenes.
- Provider downloads can still be affected by external rate limits or network instability.
- The user interface needs continued refinement to make artifact inspection clearer.

Possible Improvements

- Deploy the mask service on an OCI GPU instance while keeping the local UI and API workflow unchanged.
- Add a host-native MPS runner for local Mac acceleration when the dependency stack is stable.
- Improve UI visualization for RGB layers, cloud probability and water mask overlays.
- Add more stage-specific metrics, including model tile count, accelerator type and per-scene timing.
- Extend validation with representative water-rich, cloud-rich and multi-tile scenarios.

Conclusion

The validation phase confirms the value of the modular architecture and reveals the main optimization priority. The pipeline itself can orchestrate the required stages, but precise and fast water masking needs to run where the OmniWater model can access a real accelerator.

Confidential - Oracle Restricted \Employees Only

General Conclusion and Perspectives

This end-of-studies project focused on the design and implementation of a modular satellite imagery acquisition and processing pipeline. The work transformed a fragmented workflow into a service-oriented platform that can preview provider products, launch asynchronous jobs, download scenes, normalize outputs to Zarr, apply cloud and water masks and expose artifacts through a traceable lifecycle.

The main contribution is architectural as much as technical. By separating UI, API, worker, Zarr conversion and masking responsibilities, the platform becomes easier to maintain, test, deploy and extend. This separation also makes it possible to place heavy stages on specialized infrastructure without changing the user-facing workflow.

Future work should focus on accelerator-backed water masking, stronger UI visualization of results, broader validation across providers and tile configurations, and cloud deployment patterns that keep large raster data close to compute resources. These improvements would move the platform closer to operational use for Digital Government geospatial services.

Webography

[1] Oracle Corporation, official documentation and corporate information.

[2] Oracle Cloud Infrastructure documentation.

[3] European Space Agency, Sentinel-2 user documentation.

[4] Copernicus Data Space Ecosystem documentation.

[5] U.S. Geological Survey, Landsat Collection 2 and M2M API documentation.

[6] Zarr Developers, Zarr format documentation and specification.

[7] FastAPI documentation.

[8] Streamlit documentation.

[9] Rasterio, Xarray and PyProj documentation.

[10] NimbusChain internal repository documentation: README, API reference, Zarr notes and runbook.

Confidential - Oracle Restricted \Employees Only